Online Deployment and Evaluation of Consumption-Aware Memory Allocation for Scientific Workflows

Nagarjuna Chowdary Kandimalla, Bhanu Prakash Vangala, and Tanu Malik
Department of Electrical Engineering and Computer Science
University of Missouri
Columbia, MO 65211, USA
{nkmh5@umsystem.edu, bv3hz@missouri.edu, tanu@missouri.edu}

Abstract*—Scientific workflow schedulers must reserve memory before a task starts: insufficient reservations risk execution failures, while excess reservations waste resources. Consumption-Aware Memory Prediction (`CAMP`), introduced in our prior WORKS paper, uses historical data-consumption and memory measurements to predict the memory required by subsequent tasks. This study examines whether deploying `CAMP` online can limit memory waste while maintaining reliable execution on previously unseen workflow inputs. We describe its Nextflow integration and task-filtered eBPF measurement of explicit reads and file-backed page-fault activity, explaining how completed observations inform later allocations. The evaluation separates the contributions of read and memory-mapped access measurements, historical consumption features, and predicted consumption through controlled ablations. We also examine selective auditing's effect on downstream allocations and compare estimated allocation savings with auditing overhead. Coverage and resource use are evaluated together. As existing deployment evidence, a live Bowtie2 cold-to-warm evaluation on repeated inputs, with 6,000 logical tasks per run, shows a 71.96% reduction in total requested memory in the warm run; initial allocations cover the recorded peak memory for 99.78% of warm-run tasks. An additional evaluation on previously unseen inputs will compare frozen and online-updated models as execution history accumulates. Together, these experiments assess whether consumption feedback justifies its measurement cost and supports reliable allocation beyond previously observed inputs. *Quantitative results for the unseen-input evaluation are pending in this working draft.

***Index Terms*—scientific workflows, online memory allocation, consumption feedback, eBPF, selective auditing**

I. Introduction

Scientific workflows execute many tasks whose memory requirements vary with their inputs and processing behavior. Workflow engines must request memory before these tasks run. An insufficient reservation can cause an out-of-memory (OOM) failure and a retry; an excessive reservation occupies capacity that could serve other tasks. Predictors based on task metadata and completed executions seek to balance these costs [1]–[3]. Their practical value depends on both allocation safety and the resources reserved to achieve it.

Prelaunch information only partially describes memory demand. A task may read a small fraction of a large staged index, or access file-backed data through memory-mapped page faults rather than explicit read calls. Conversely, a streaming task may process many bytes while retaining little data in memory. Consumption measurements therefore provide contextual evidence rather than a direct measure of peak resident memory. The pending task's consumption is also unavailable at allocation time; only completed observations can inform its request.

Our prior WORKS paper introduced Consumption-Aware Memory Prediction (`CAMP`) [4]. It combines static task features and historical peak memory with consumption measurements from audited tasks. An auxiliary model predicts consumption before launch; quantile models and residual calibration then produce a memory request. Task-attributed eBPF measurements and completed execution outcomes supply feedback for later allocations. The present paper builds on this architecture to study a practical deployment question: whether consumption feedback can reduce wasted memory while maintaining reliable execution as workflows encounter previously unseen inputs.

The earlier CAMP study compares prediction with and without consumption features and evaluates Sizey, selected ablations, and selective-auditing results [4]. Existing deployment results in the CAMP artifact include a live Bowtie2 cold-to-warm evaluation [5]. Across 6,000 logical tasks per run, total requested memory falls by 71.96% in the warm run, with initial allocations covering recorded peak memory for 99.78% of tasks. This allocation coverage is distinct from execution success. These results establish the feasibility of using accumulated history on repeated inputs. Assessing the value and cost of that feedback, and its behavior on unseen inputs, requires additional controlled comparisons.

Three considerations guide this evaluation. First, the value of consumption feedback depends on both measurement and prediction. Explicit reads and file-backed page-fault activity capture different access behavior; historical consumption and predicted consumption supply different model inputs. Separating these effects requires fixed comparison conditions.

Second, auditing incurs execution cost, while its observations can inform only later tasks. Selecting informative tasks is therefore insufficient by itself: the resulting allocation

savings must be considered alongside the cost of collecting their measurements. Coverage and reservation cost must also be assessed together, since fewer underallocations may result from larger requests. Development-only tuning and a common prelaunch information boundary keep the component studies comparable.

Third, repeated execution and adaptation to new inputs test different properties. A frozen model and history assess what prior observations already support. Online updating additionally incorporates outcomes from the new executions. Comparing these modes from the same initial state on the same unseen inputs provides a controlled test of adaptation. In live execution, allocation coverage must be accompanied by observed failures and retries, since a request below recorded peak memory does not by itself establish an OOM event.

Our evaluation addresses three aspects of practical memory allocation:

- *Measurement and features:* eBPF task filtering and Nextflow integration, explicit reads alone versus reads with file-backed page-fault activity, and four feature variants that add historical consumption, predicted consumption, or both to static features and memory history;
- *Allocation benefit and cost:* downstream allocations under selective auditing, estimated memory-time savings relative to auditing overhead, with coverage and reservation cost evaluated together; and
- *Online deployment and adaptation:* the existing Bowtie2 feedback loop and a controlled evaluation protocol for frozen versus online-updated allocation on previously unseen inputs.

CAMP's architecture and previously reported results remain prior work. The extension examines when its additional information is useful and whether the allocation benefit justifies the measurement cost.

Section II states the allocation problem and information constraints. Section III summarizes CAMP and its relationship to WORKS. Sections IV and V describe the experimental methods and results, followed by related work (Section VI) and conclusions (Section VII).

II. Problem Formulation

We retain the prelaunch allocation problem introduced in WORKS [4]. A workflow comprises named processes, each of which instantiates tasks for different samples or input partitions. Before task t starts, the allocator issues an initial memory request $\hat{y}_t$. Its static feature vector $\mathbf{s}_t$ contains workflow and process metadata, software and system context, resource settings, and staged input size. Execution subsequently reveals peak resident set size (RSS), y_t, and duration d_t. If the task is audited, it also supplies a consumption measurement c_t. Neither y_t nor c_t is available for that task's initial allocation.

Let $\mathcal{P}(t)$ contain tasks whose outcomes have been recorded before the allocation decision for t. Peak-memory history $\mathbf{h}_{m,t}$ summarizes compatible observations from this set; consumption history $\mathbf{h}_{c,t}$ uses its audited subset. The prelaunch decisions are

$$\hat{y}_t = f(\mathbf{s}_t, \mathbf{h}_{m,t}, \mathbf{h}_{c,t}), \qquad q_t \in \{0, 1\}, \tag{1}$$

where $q_t = 1$ selects auditing. A budget $B \in (0, 1]$ limits the audited fraction over the specified evaluation horizon. Auditing task t cannot improve the request already issued to it; its measurements can inform only later decisions. In live execution, availability follows completion and ingestion times, rather than launch order. Offline evaluation must state the history construction and outcome-revelation protocol used to approximate this constraint.

The allocation objective is to limit underallocation while avoiding unnecessary reservations. For N evaluated tasks, initial-request coverage is

$$\mathrm{Cov} = \frac{1}{N} \sum_{t=1}^{N} \mathbf{1}[\hat{y}_t \geq y_t], \tag{2}$$

where $\mathbf{1}[\cdot]$ is the indicator function. This is the *first-attempt coverage* measure used in WORKS: retry increases do not enter $\hat{y}_t$. A request below a recorded peak is an allocation diagnostic, not proof of an observed OOM failure. Live studies therefore report task completion, failures, and retries separately.

Coverage is considered alongside aggregate requests $\sum_t \hat{y}_t$ and unused memory-time,

$$W = \sum_{t \in \mathcal{D}} \max(\hat{y}_t - y_t, 0)\, d_t, \tag{3}$$

where $\mathcal{D}$ identifies tasks with valid duration measurements. Using memory in GiB and duration in hours gives W in GiB-hours. This peak-based measure quantifies reservation above observed peak demand; it is not an integral of instantaneous unused RSS. Aggregate requests likewise do not represent simultaneous cluster usage. Missing durations and retry accounting must be stated rather than treated as measured zero cost. When estimating net savings, auditing overhead must be charged in compatible units and distinguished from prediction or retraining cost.

For unseen-input evaluation, a frozen condition holds the model, history, and calibration state fixed. An online condition starts from the same state and incorporates eligible completed outcomes. This comparison separates allocation from existing knowledge from adaptation during the new workload. Both conditions exclude the current task's outcomes from its prelaunch decision.

III. CAMP Background and Relationship to WORKS

CAMP's measurement, prediction, and selective-feedback architecture was introduced in WORKS [4]. We summarize the existing design and its implementation to identify the components examined in this study.

A. Measurement and compatible history

The eBPF tracer is attached to a task launcher and tracks descendants through process-fork events. Kernel handlers discard events from untracked execution contexts, preventing unrelated tasks on the same node from contributing to that task's measurements. For explicit reads, the implementation associates descriptors with observed file opens and accumulates positive return values from `read` and `pread64`. These are returned bytes, rather than requested buffer lengths; descriptor attribution depends on the file operations observed by the tracer.

A successful `mmap` call establishes a virtual mapping but does not show how much mapped data is accessed. CAMP instead instruments the `filemap_fault` entry point for file-backed page-fault activity. The retained tracer adds 4,096 bytes per attributed event. The resulting consumption signal is explicit-read bytes plus this page-fault byte estimate. It does not deduplicate pages or certify that each entry event completed a successful fault; it is therefore not an exact count of unique bytes consumed, physical disk traffic, or resident memory. This distinction motivates the read-only versus read-plus-mmap comparison. A task that streams data and one that retains an index may have similar consumption but different memory peaks.

Completed observations are organized into compatibility scopes. I_0 and I_1 match task-instance/split-group and input identity, respectively; I_2 selects nearby static-feature signatures; I_3 pools process history. These scopes preserve workflow, version, and system context. I_4 broadens to workflow history and I_5 to system-compatible history. Support thresholds select a usable scope, while summaries from multiple scopes remain features. History scopes identify relevant observations; prediction-model selection is a separate mechanism.

B. Consumption prediction and memory allocation

The Access baseline, denoted A+P in the artifact, uses static features and peak-memory history. CAMP adds historical consumption and a prelaunch consumption estimate:

$$\mathbf{x}_t^{\mathrm{AP}} = [\mathbf{s}_t, \mathbf{h}_{m,t}], \tag{4}$$

$$\hat{c}_t = g(\mathbf{s}_t, \mathbf{h}_{m,t}), \tag{5}$$

$$\mathbf{x}_t^{\mathrm{CAMP}} = [\mathbf{s}_t, \mathbf{h}_{m,t}, \mathbf{h}_{c,t}, \hat{c}_t]. \tag{6}$$

The estimator g learns from audited outcomes. Historical consumption $\mathbf{h}_{c,t}$ describes earlier executions, whereas $\hat{c}_t$ estimates the pending task's consumption. The four feature variants retain A+P alone, add either signal separately, or include both. In the offline training pipeline, cross-fitted proxy predictions supply the training feature: each held-out group is predicted by a model fitted without that group. Measured current-task consumption is not substituted for this prediction.

LightGBM ensembles estimate peak-memory quantiles at $\{0.50, 0.90, 0.95, 0.99, 0.995\}$. The process-tail configuration combines a global central estimate with workflow–process upper-tail models and a global fallback; global-tail configurations also exist in the artifact. Feature comparisons must therefore hold the prediction configuration fixed or explicitly identify configuration differences. Changing features and model selection together cannot isolate the contribution of a feature.

C. Calibration and selective feedback

A development-selected policy chooses a base estimate b_t and a residual quantile. Hierarchical calibration uses log residuals $r_\tau = \log(y_\tau / b_\tau)$ and blends a supported residual scope with a broader parent. Residual support determines this scope independently of the history-feature scope. The correction ρ_t produces

$$\hat{y}_t = \lceil b_t \exp(\max(0, \rho_t)) \rceil_{\mathrm{MiB}}. \tag{7}$$

The adjustment is one-sided and rounded to whole mebibytes. Cross-fitted predictions provide development residuals; subsequently available outcomes support online updates. Calibration remains empirical and provides no unconditional coverage guarantee on shifted inputs.

The selective policy allocates an audit budget using prelaunch risk, information, and discovery signals. Peak-memory outcomes can update history after completion; consumption outcomes enter only when measured. The deployment mechanisms are distinct from this policy: the generic Nextflow audit overlay supports process-name filtering and periodic task selection, while the retained live Bowtie2 configuration audits every task. Thus, the live run does not by itself demonstrate selective-auditing gains.

In the artifact, Nextflow configuration overlays add instrumentation to unmodified nf-core workflow sources. A launch wrapper pauses the target, attaches the tracer, waits for readiness, and resumes execution. Bowtie2's allocation integration additionally defines process-specific memory closures that pass task metadata to the prediction service and return requests to Nextflow for Slurm submission. Completion hooks record outcomes, and an asynchronous learner publishes updated model versions for later predictions. The retained launcher requests model updates after each 128 eligible outcomes; history ingestion and model retraining are separate operations. These additions preserve analytical tool commands but require workflow-specific metadata bindings, deployment configuration, and Linux hosts with BCC/eBPF support and tracing privileges. They are not a transparent allocator for every workflow engine.

D. Prior evidence and extension boundary

WORKS already reports the architecture, Access and Sizey comparisons, selected feature and mmap ablations, and selective-auditing results [4]. The retained Bowtie2 runs provide existing deployment evidence [5]. The present study develops the component analysis, examines downstream allocation benefit relative to audit cost, and specifies a controlled extension to unseen inputs. Reanalysis of existing measurements is distinguished from new experiments. Routing, calibration, and compatibility scopes are described as background; separate ablation sweeps of these mechanisms are outside the present scope.

IV. Experimental Method

A. Populations and comparison conditions

We analyze retained predictions and execution records in the CAMP artifact [5]; no new workflow execution is implied by replotting these measurements. The feature and mmap studies use 33,516 tasks from GATK, Minimap2, Sarek, Seqinspector, Taxprofiler, and Viralmetagenome. The fixed seed-1996 split assigns 28,490 tasks to development and 5,026 to evaluation, using chronological ordering within each workflow. Four feature views use static information and peak-memory history (A+P), add consumption history ($\mathbf{h}_c$), add predicted consumption ($\hat{c}$), or include both (full CAMP).

The retained LightGBM ensembles and development-selected allocation policies are preserved. Table II compares each feature view under the same global or process-tail configuration; it is a control for feature interpretation, not a new routing-policy sweep. The original endpoint comparison uses global A+P and process-tail CAMP. Published requests and their retained reproduction are reported separately. The mmap study uses paired task identities and peaks; each arm selects its own allocation policy on development data, without test-time request scaling. MdAPE denotes the median absolute percentage error of the median peak prediction, not the error of the final reservation.

B. Selective feedback and audit accounting

The downstream replay uses a separate 30,478-task cohort: 9,141 development and 21,337 evaluation tasks. Five fixed batches reveal recorded outcomes at batch completion. Selective and uniform-random auditing use equal per-workflow counts at budgets of 5, 10, 20, 30, and 50%, with five sampling seeds (1996–2000) and fixed model seeds. The primary evaluation covers batches 2–5: 17,067 tasks after the first update. This is a batched replay, not a reconstruction of live concurrency. Shared initial audited history is excluded from incremental audit budgets.

A separate accounting study holds the published 5,026-task allocations fixed and charges positive paired auditing overhead:

$$\begin{aligned} C_B &= \sum_{t:q_t=1} \hat{y}_t \max(d_t^{\text{audit}} - d_t^{\text{base}}, 0), \\ G_B &= (W_{\text{AP}} - W_{\text{CAMP}}) - C_B. \end{aligned} \tag{8}$$

Memory and time are converted to GiB and hours. All tasks have paired audit timings, but allocation-saving runtimes exist for only 4,577 tasks; the 449 missing runtimes retain zero weight in the published saving. G_B excludes prediction, retraining, and retry costs and does not estimate the allocation improvement caused by each budget. Recorded 95% intervals use paired grouped bootstrap for downstream differences and bootstrap of five seed-level means for audit accounting.

C. Live deployment

The retained Nextflow/Slurm deployment uses 12 task workers and a thirteenth node for control, prediction, and learning, each with 32 vCPUs and about 256 GB RAM. Bowtie2 processes 38 paired ENA runs through seven workflow processes, producing 6,000 logical tasks per execution. The cold run starts without Bowtie2 history, updates during execution, and seeds a fresh warm execution of the same inputs; Nextflow's task cache is not reused. Both runs audit all tasks. Logical-task metrics use initial requests and final recorded peaks/runtimes; attempt tables separately retain failures and retries. These runs do not compare frozen and online-updated allocation on unseen inputs.

TABLE I
MMAP ABLATION ON THE SAME 5,026 TASKS. NATIVE DEVELOPMENT-SELECTED REQUESTS; NO REQUEST SCALING.

Signal	Underalloc.	Requests (GiB)	MdAPE (%)
Read-only	32	41,853	22.04
Read + mmap	30	34,271	26.51

V. Results

The following results distinguish published WORKS endpoints from further analysis of retained measurements. Unseen-input results remain pending.

A. Memory-mapped access and feature contributions

Table I details the read-only versus Read + mmap comparison briefly reported in WORKS [4]. Adding the mmap signal reduces underallocations from 32 to 30 and total requests by 18.11%. Median prediction error nevertheless increases from 22.04% to 26.51%. Thus, a better reservation tradeoff need not coincide with lower median prediction error; the two-task difference alone is not evidence of a robust advantage across datasets or random splits.

WORKS reports 87 underallocations for A+P and 31 for CAMP, with total requests of 26,067.45 and 34,277.55 GiB [4]. The retained reproduction gives 85 and 30; these are distinct request realizations, not replacements for the published numbers. Table II expands the four feature combinations using configuration controls. Under global prediction, full CAMP gives 74 underallocations versus 85 for A+P, with 2.06% more requested memory. Under process-tail prediction, A+P attains fewer underallocations (24 versus 30) but requests 603,467 versus 34,271 GiB. The full feature set therefore does not dominate every safety metric. Relative to consumption history alone under process-tail prediction, adding $\hat{c}$ reduces underallocations from 46 to 30 and requests by 57.61%, while MdAPE increases from 21.32% to 26.51%. These comparisons support evaluating feature value jointly with reservation cost.

B. Selective auditing and estimated net saving

At the primary 20% budget, the retained downstream replay gives 265.8 versus 305.8 mean underallocations for selective and random auditing: 13.1% fewer, with 2.4% more requested memory. This preliminary result was also summarized in WORKS [4]. Figure 1(a) shows that the direction varies with budget. At 20%, the paired 95% interval for the rate difference is $[-0.416, 0.007]$ percentage points, which includes zero. The

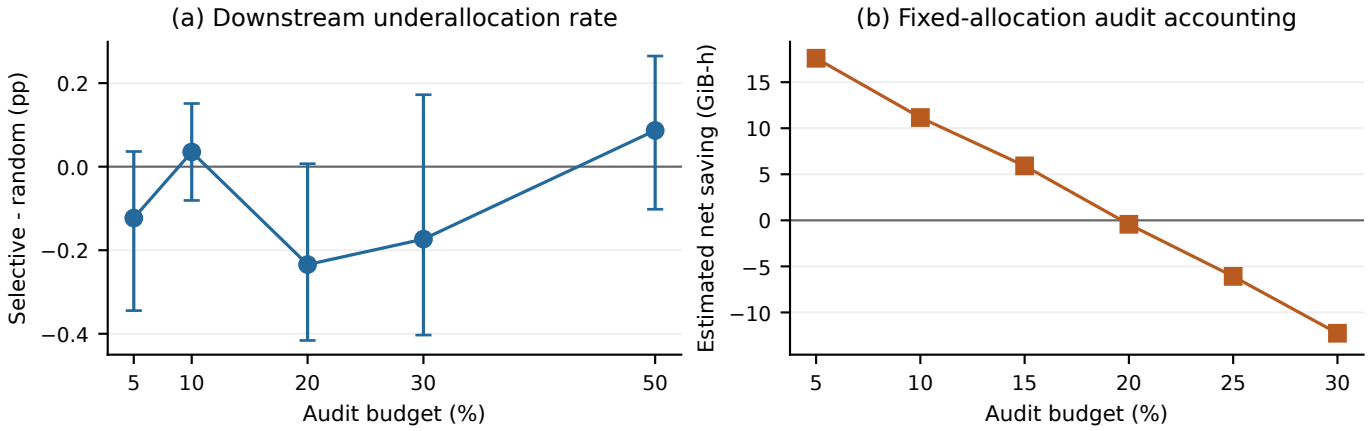


Fig. 1. Two separate auditing studies. (a) Native downstream underallocation-rate difference over 17,067 tasks after the first update; negative values favor selective auditing. Bars are recorded paired 95% bootstrap intervals. (b) Estimated net saving on 5,026 tasks with published allocations fixed (5–30% budgets shown); bars are 95% bootstrap intervals of five seed-level means. Allocation savings in (b) are not caused by the budget choices in (a).

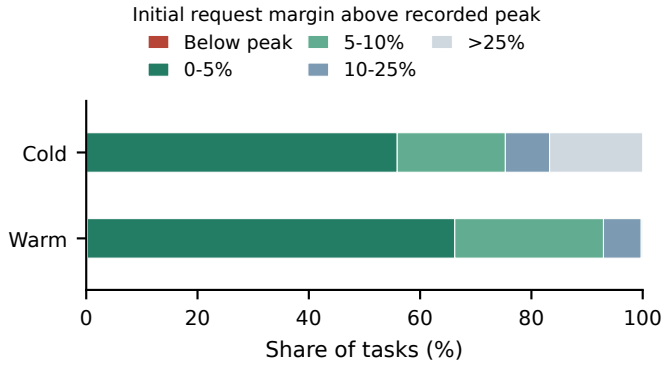


Fig. 2. Initial allocation margins in the retained Bowtie2 runs. The warm run concentrates requests near recorded peaks; repeated inputs do not test unseen-input adaptation.

TABLE II
FOUR FEATURE VARIANTS WITH CONFIGURATION HELD FIXED (5,026 TASKS). U: UNDERALLOCATION COUNT; R: AGGREGATE REQUESTS IN THOUSANDS OF GIB.

	Global		Process-tail	
Feature view	U	R	U	R
A+P	85	26.08	24	603.47
A+P+$\mathbf{h}_c$	81	26.34	46	80.85
A+P+$\hat{c}$	105	26.45	52	123.96
Full CAMP	74	26.61	30	34.27

five sampling repetitions do not establish uniform improvement or robustness across model seeds.

The fixed-allocation saving is 23.596 GiB-hours before audit cost. Figure 1(b) gives net estimates of 17.597, 11.161, and 5.913 GiB-hours at 5, 10, and 15% budgets. At 20%, the estimate is -0.429 with interval $[-1.077, 0.346]$ GiB-hours. At 25, 30, 50, and 100%, net estimates are -6.087, -12.259, -42.938, and -116.084 GiB-hours. These values identify when the fixed saving covers measured audit cost; they are not end-to-end deployment savings.

TABLE III
EXISTING BOWTIE2 COLD-TO-WARM DEPLOYMENT ON REPEATED INPUTS. EACH RUN CONTAINS 6,000 LOGICAL TASKS ACROSS SEVEN PROCESSES.

Metric	Cold	Warm
Initial requests (GiB)	898.39	251.93
Recorded peak sum (GiB)	238.50	238.82
Unused memory-time (GiB-h)	23.954	0.018
Initial-request coverage (%)	99.93	99.78
Requests below recorded peak	4	13
Completed logical tasks	6,000	6,000
Execution attempts	6,016	6,015
Retries	16	15
Recorded first-attempt OOMs	0	0

C. Existing Bowtie2 online deployment

Table III reports the retained live runs. Warm execution reduces aggregate initial requests by 71.96%, from 898.39 to 251.93 GiB, while observed peak totals remain approximately 239 GiB. Unused memory-time decreases from 23.954 to 0.018 GiB-hours. This logical-task measure excludes the cost of failed attempts and the separately measured auditing and learning overheads.

Figure 2 shows the proportion allocated between the peak and 10% above it increasing from 75.25% to 92.73%. Requests below recorded peak increase from four to thirteen, giving warm coverage of 99.78%. All 6,000 tasks eventually complete in both runs, with 16 and 15 retries. The records flag no first-attempt OOMs; below-peak requests must therefore not be relabeled observed OOM failures. This experiment supports feedback reuse on repeated inputs; it does not establish transfer to unseen data.

D. Remaining evaluation

Results for the unseen-input frozen-versus-online comparison are **?** (pending). Prediction latency and retraining resource

costs also remain **?**; no claim of total system-level net benefit is made from the audit-only accounting.

VI. Related Work

Online workflow allocation. Witt et al. [1] connect execution measurements, resource prediction, and batch scheduling through feedback. Sizey [3] trains multiple predictors and selects models online using completed-task observations. Ponder [2] adapts its sizing rule to available history and the relationship between input size and peak memory, combining regression with historical bounds and fallback requests. CAMP incorporates consumed-data history and predicted consumption alongside task attributes and historical memory measurements.

Distributional prediction. NGBoost [6] predicts parameters of a conditional outcome distribution, enabling uncertainty estimates beyond a point prediction. CAMP instead estimates selected memory quantiles and applies empirical residual calibration. Neither a distributional output nor a lower median prediction error alone establishes a better allocation policy: the reservation quantile and calibration determine the coverage–cost tradeoff. Our component studies assess coverage and reservation cost together when examining the contribution of consumption features.

Relationship to CAMP. WORKS introduced consumption-aware prediction and selective auditing [4]. This paper treats that architecture as prior work and examines its operational consequences through retained component comparisons, downstream feedback, audit-cost accounting, and live deployment records. The planned unseen-input study addresses a further question that the repeated-input deployment cannot resolve.

VII. Conclusion

Draft section: pending extension work.

The conclusion will be written after the extension experiments establish their findings, reservation-cost tradeoffs, and limits. No final effectiveness claim is made in this working draft.

Acknowledgment

This work is supported by NASA under grant NASA-AIST-21-0095.

References

[1] C. Witt, D. Wagner, and U. Leser, "Feedback-based resource allocation for batch scheduling of scientific workflows," in *2019 International Conference on High Performance Computing and Simulation (HPCS)*. Dublin, Ireland: IEEE, 2019, pp. 761–768.

[2] F. Lehmann, J. Bader, N. De Mecquenem, X. Wang, V. Bountris, F. Friederici, U. Leser, and L. Thamsen, "Ponder: Online prediction of task memory requirements for scientific workflows," in *2024 IEEE 20th International Conference on e-Science (e-Science)*. Osaka, Japan: IEEE, 2024, pp. 1–10.

[3] J. Bader, F. Skalski, F. Lehmann, D. Scheinert, J. Will, L. Thamsen, and O. Kao, "Sizey: Memory-efficient execution of scientific workflow tasks," in *2024 IEEE International Conference on Cluster Computing (CLUSTER)*. Kobe, Japan: IEEE, 2024, pp. 370–381.

[4] N. C. Kandimalla, B. P. Vangala, and T. Malik, "CAMP: Consumption-aware memory prediction for scientific workflows," in *Workshop on Workflows in Support of Large-Scale Science (WORKS)*, 2026, accepted for publication; to appear.

[5] N. Kandimalla and T. Malik, "CAMP experimental artifacts and reproduction package," https://anonymous.4open.science/r/IEEE_WORKS_26_ARTIFACT-B98F/README.md, 2026, accessed: 2026-08-08.

[6] T. Duan, A. Avati, D. Y. Ding, K. K. Thai, S. Basu, A. Y. Ng, and A. Schuler, "NGBoost: Natural gradient boosting for probabilistic prediction," in *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 2690–2700.